

경력기술서

김가을 Backend Developer · Java / Spring

010-8790-1519 · gayulz@kakao.com
github.com/gayulz · yurizzly.tistory.com · gayul.vercel.app

프로젝트 1. 그룹 임직원 통합 인증 포털 — 관계사 비밀번호 재설정 · 로그인 2차 인증(2FA) 신규 개발

수행 기간	2026.06 ~ 진행 중 (개발기 검증 완료 · 상용 배포 예정)	수행 인원	백엔드 1인 전담
소속 / 연계	포커스원 / 그룹사 운영팀 연계 (S그룹 통합 로그인·본인확인 플랫폼)		
담당 역할	설계 · 구현 · 단위 테스트 · 보안 검증 · 배포 계획 수립 전 과정 수행		
기술 스택	Java 17, Spring Boot 3.3.2, Spring Security, JPA, MyBatis, MariaDB, JUnit 5, Mockito, Gradle(멀티모듈), Git, Linux(페쇄망)		
프로젝트 개요	그룹 임직원이 사용하는 통합 로그인·본인확인 플랫폼의 사용자 모듈에서, 그룹 관계사(호텔) 임직원 대상 비밀번호 재설정 전체 플로우와 브라우저 로그인 2차 인증(2FA) 을 신규 개발. 앱·웹뷰·브라우저 3개 진입 채널 을 단일 서버 플로우로 통합하고 그룹 인증 게이트웨이(AD)와 실연동 (신규 Java 클래스 17종 · 약 6,700라인)		

주요 수행 업무

인증 · 보안 설계 (시큐어 코딩)	- 인증 통과 여부를 서버 세션에만 기록하고 변경 API가 이를 검증하는 인증 게이트를 설계, 화면 흐름을 우회한 API 직접 호출·URL 직접 입력 접근을 서버에서 차단 - SMS/이메일 6자리 OTP(SecureRandom) 기반 2차 인증 구현 — 발송 5분 내 5회 제한 + 검증 5회 오입력 시 만료의 이중 rate-limit으로 무차별 대입 차단. 해당 취약점은 OWASP 기준 자체 보안 점검에서 HIGH 등급 2건으로 직접 식별해 상용 적용 전 해결 - 발송 대상 번호·이메일을 클라이언트 입력이 아닌 DB 등록값으로만 강제하고, 요청에 연락처 파라미터가 실려 오면 번호 시도로 간주해 발송을 거부하는 파라미터 변조 감지 로직 구현 - 비밀번호 정책(자릿수·계정ID 포함 금지·특수문자 화이트리스트)을 프론트와 별개로 서버에서 재검증하는 2중 방어 구축 - 로그 3채널에 분산된 비밀번호류 마스킹을 단일 유틸로 통일하는 과정에서 마스킹 정규식의 ReDoS 취약점을 발견(48KB 입력 시 2.6초 지연), 패턴 유계화로 0.63ms까지 개선
외부 연동 · 채널 통합	- 그룹 인증 게이트웨이(AD)의 비밀번호 변경 API 실연동 — 응답 코드별(성공/오입력/계정 잠김) 화면 라우팅 계약 설계, 프로토콜 불일치로 인한 HTTP 400 장애를 curl 재현으로 원인 규명 후 해결 - 게이트웨이가 지원하지 않는 "이전 비밀번호 재사용 차단"을 기존 비밀번호 선행 검증 후 서버 비교 방식으로 구현 3개 진입 채널의 세션 부트스트랩·완료 복귀 신호(JSON 계약)를 설계, 모바일 앱팀과 인터페이스 계약 문서를 5차례 개정하며 확정 - 완료 안내 메일을 다국어(한/영/중) DB 템플릿으로 구현, @Async 전용 스레드풀(graceful shutdown 포함)을 최초 도입해 응답 흐름과 발송 분리
품질 · 테스트 (JUnit)	JUnit 5 + Mockito 단위 테스트 130여 케이스 신규 작성 — MockedStatic으로 게이트웨이 실통신을 mock 처리해 응답 코드별(성공/실패/오류/무응답) 분기 시나리오 전수 검증 - QA·앱팀 E2E 테스트용 게이트웨이 스텝 모드(상용 fail-closed)를 설계·제공하고, 실연동 검증 완료 후 스텝 코드를 전량 제거해 상용 소스 정화
배포 · 문서화	- 대상 관계사를 설정 기반 화이트리스트로 설계해 관계사 확대 시 코드 수정 없이 대응, 장애 시 2FA만 단독 revert 가능한 2단계 머지 배포 전략과 런타임 기능 스위치 병행 설계 - 인증 플로우맵·진입점 분석·연동 계약·배포 파일 인벤토리 등 설계·의사결정 문서 100여 건 작성, 게이트웨이트임·앱팀·QA 협업과 팀 보고에 활용
주요 성과	단위 테스트 130+ 신규 작성 · OWASP HIGH 취약점 2건 자체 식별 후 상용 적용 전 해결 · 마스킹 ReDoS 2.6초 → 0.63ms 개선 (48KB 입력 실측)

프로젝트 2. 사내 모바일 앱스토어 시스템 Spring Boot 3.x 전환 및 현대화

수행 기간	2025.11 ~ 2026.05 (약 7개월) · 운영 배포 완료, 이후 운영·고도화 지속	수행 인원	1명 (개발·운영 전 과정 수행)
소속 / 연계	포커스원 / 그룹사 운영팀 연계		
담당 역할	프레임워크 마이그레이션 총괄, 멀티모듈 아키텍처 재설계, 인증·보안 체계 전환(Spring Security 6.x · 사내 SSO 연동), 운영 배포 및 안정화		
기술 스택	Java 17, Spring Boot 3.3.2, Spring Security 6.x, MyBatis, JPA/Hibernate, MariaDB, Gradle 8.8, Git / 운영: Linux(페쇄망), 개발: 페쇄망 VDI		
프로젝트 개요	S그룹 임직원 약 11만 명이 사용하는 사내 모바일 앱스토어의 레거시 백엔드(Java 1.7 / 구형 Spring, 클래스 151개·Mapper XML 19개·JSP 66개)를 Java 17 / Spring Boot 3.3.2로 전환, 2026.05 운영 배포 완료 . 단순 버전 이식이 아닌 아키텍처 재설계·보안 강화·코드 품질 개선 병행		

주요 수행 업무

계층별 단계 전환	· "빌드와 기동이 되는 상태를 절대 깨지 않는다"는 원칙으로 인프라 → 보안·인증 → 비즈니스 로직 → 뷰 순 한 계층씩 전환, 원인 불명 오류로 인한 중단 없이 전 구간 완료 · 페쇄망 제약 대응을 위해 클라이언트 IP·세션ID가 전 로그에 자동 기록되는 커스텀 Logback 구조를 최우선 구축 — 서버 접속 없이 로그만으로 원인 특정, 버그 수정 주기 3일 → 1일 단축 · 호환성 이슈 53건을 Jakarta 패키지·Security 설정·MyBatis 바인딩·뷰 경로·빌드 의존성 유형으로 분류·기록하며 전량 해결
데이터 설계 · SQL	· 분산된 로그인 이력 테이블을 단일 테이블로 통합, 운영 쿼리 WHERE 절 패턴 전수 분석 후 핵심 조회 경로별 복합 인덱스 직접 설계(IP, RESULT, LOG_DT — 동등 조건 → 범위 조건 순 배치) · 대표 쿼리 EXPLAIN 확인으로 Full Table Scan → Index Range Scan 전환 검증, 조회·페이징·집계 쿼리 직접 작성 · 단건 조회 전제 Repository 메서드가 운영 중복 데이터로 IncorrectResultSizeDataAccessException 유발 → List 조회로 장애 즉시 차단, 중복 원인은 정렬 기준 명시·유니크 제약 검토 과제로 분리해 임시 조치와 근본 해결을 구분 기록
트랜잭션 · 데이터 접근	· "복잡한 동적 쿼리는 MyBatis, 단순 CRUD는 JPA" 하이브리드로 구성하고 트레이드오프 근거를 문서화 · 두 기술이 동일 DataSource·트랜잭션 경계를 공유하도록 설정 정리, 예외 발생 시 양쪽 변경 동시 롤백 여부 직접 검증, open-in-view: false로 예측 불가 지연 로딩 차단 · 필드 주입 → 생성자 주입 전환으로 드러난 서비스 간 순환 참조를 조회 책임 분리로 해소
보안 · 운영 안정화	· 구형 XML 인터셉터 인증을 Spring Security 6.x SecurityFilterChain 기반 커스텀 필터로 재설계, 로그인 실패 이력 기반 IP·사번 이중 차단 정책 설계 · 사내 SSO(SAML) 인증 개편 — Session Fixation 방지용 세션 재생성 시 내부 검증 플래그가 사라지는 문제를 진입 경로 정보화와 콜백 시 플래그 재설정으로 해결 · Spring Boot 내장 Tomcat 전환 + 앞단 Apache(AJP) 연동 유지로 인프라 변경 최소화하며 운영 배포, 배포 중 발견된 레거시 잠재 결함(NPE·리다이렉트 오류) 즉시 수정 · 파일 다운로드 중 연결 종료 I/O 예외가 ERROR로 기록돼 모니터링 오경보 유발 → 전역 예외 핸들러에서 원인별 분류 처리로 해소
주요 성과	호환성 이슈 53건 전량 해결 (유형별 분류 기록 기준) · 버그 수정 주기 3일 → 1일 단축 · 빌드 시간 28% 단축 (전환 전후 빌드 로그 비교 기준)

그 외 수행 업무 (2024.04 ~)

노후 라이브러리 교체 (2024~2025)	API 통신 방식·서비스 로직 재설계, 앱 식별값 검증 로직 및 XSS·SQL Injection 방어 필터 서버단 구현
상시 운영	관리자·사용자 페이지 백엔드 기능 개발, 장애 대응 (Apache 액세스 로그·DB 쿼리 분석 기반 원인 추적), 2026.05 운영 배포 이후 전환 버전 운영·고도화 지속